-LAPORAN PEMROGRAMAN BERBASIS OBJECT

JOBSHEET 12



Nama:Daffa Dwi Nugroho

NIM : 254107020140

Kelas: TI 2-A

Link Github: https://github.com/Dapa-yap/Praktikum-Berbasis-Objek.git

D-IV TEKNIK INFORMATIKA

JURUSAN TEKNOLOGI INFORMASI

TAHUN AJARAN 2026

JOBSHEET 12

4. Percobaan 1 – Bentuk dasar polimorfisme

4. 1. Langkah Percobaan

1. Buat class Employee

```java
package P1;

public class Employee {
    protected String name;

    public String getEmployeeInfo() {
        return "Name = " + name;
    }
}
```

2. Buat interface Payable

```java
package P1;

public interface Payable {
    public int getPaymentAmount();
}
```

3. Buat class InternshipEmployee, subclass dari Employee

```java
package P1;

public class InternshipEmployee extends Employee {
    private int length;

    public InternshipEmployee(String name, int length) {
        this.length = length;
        this.name = name;
    }

    public int getLength() {
        return length;
    }

    public void setLength(int length) {
        this.length = length;
    }

    @Override
    public String getEmployeeInfo() {
        String info = super.getEmployeeInfo() + "\n";
        info += "Registered as internship employee for "
                + length + " month/s\n";
        return info;
    }
}
```

4. Buat class PermanentEmployee, subclass dari Employee dan implements ke Payable

```java
package P1;

public class PermanentEmployee extends Employee
        implements Payable {

    private int salary;

    public PermanentEmployee(String name, int salary) {
        this.name = name;
        this.salary = salary;
    }

    public int getSalary() {
        return salary;
    }

    public void setSalary(int salary) {
        this.salary = salary;
    }

    @Override
    public int getPaymentAmount() {
        return (int) (salary + 0.05 * salary);
    }

    @Override
    public String getEmployeeInfo() {
        String info = super.getEmployeeInfo() + "\n";
        info += "Registered as permanent employee with salary "
                + salary + "\n";
        return info;
    }
}
```

5. Buat class ElectricityBill yang implements ke interface Payable

```java
package P1;


public class ElectricityBill implements Payable {
    private int kwh;
    private String category;

    public ElectricityBill(int kwh, String category) {
        this.kwh = kwh;
        this.category = category;
    }

    public int getKwh() {
        return kwh;
    }

    public void setKwh(int kwh) {
        this.kwh = kwh;
    }

    public String getCategory() {
        return category;
    }

    public void setCategory(String category) {
        this.category = category;
    }

    @Override
    public int getPaymentAmount() {
        return kwh * getBasePrice();
    }

    public int getBasePrice() {
        int price = 0;

        switch (category) {
            case "R-1":
                price = 100;
                break;
            case "R-2":
                price = 200;
                break;
        }

        return price;
    }

    public String getBillInfo() {
        return "KWH = " + kwh + "\n"
                + "Category = " + category
                + "(" + getBasePrice() + " per KWH)\n";
    }
}

```

6. Buat class Tester1

```java
package P1;

public class Tester1 {

    Run | Debug
    public static void main(String[] args) {
        PermanentEmployee pEmp = new PermanentEmployee(name: "Dedik", salary: 500);

        InternshipEmployee iEmp = new InternshipEmployee(name: "Sunarto", length: 5);

        ElectricityBill eBill = new ElectricityBill(kwh: 5, category: "R-1");

        Employee e;
        Payable p;

        e = pEmp;
        e = iEmp;

        p = pEmp;
        p = eBill;
    }

}
```

4.2. Pertanyaan

1. Class apa sajakah yang merupakan turunan dari class Employee?

- InternshipEmployee
- PermanentEmployee

2. Class apa sajakah yang implements ke interface Payable?

- PermanentEmployee
- ElectricityBill

3. Perhatikan class Tester1, baris ke-10 dan 11. Mengapa e, bisa diisi dengan objek pEmp (merupakan objek dari class PermanentEmployee) dan objek iEmp (merupakan objek dari class InternshipEmploye) ?

- Karena variabel e bertipe Employee, sedangkan PermanentEmployee dan InternshipEmployee merupakan subclass dari Employee.

4. Perhatikan class Tester1, baris ke-12 dan 13. Mengapa p, bisa diisi denganobjekpEmp (merupakan objek dari class PermanentEmployee) dan objek eBill (merupakan objek dari class ElectricityBill) ?

-Karena variabel p bertipe Payable.

- pEmp bisa dimasukkan karena PermanentEmployee mengimplementasikan Payable.
- eBill bisa dimasukkan karena ElectricityBill juga mengimplementasikan Payable.

5. Coba tambahkan sintaks: p = iEmp; e = eBill; pada baris 14 dan 15 (baris terakhir dalam method main) ! Apa yang menyebabkan error?

- p = iEmp error karena InternshipEmployee tidak mengimplementasikan interface Payable.
- e = eBill error karena ElectricityBill bukan turunan dari Employee.

Jadi, objek hanya bisa dimasukkan jika memiliki hubungan inheritance atau implementasi interface yang sesuai dengan tipe variabelnya.

6. Ambil kesimpulan tentang konsep/bentuk dasar polimorfisme!

- Polimorfisme memungkinkan sebuah variabel dengan tipe superclass atau interface untuk mereferensikan objek dari berbagai subclass atau class yang mengimplementasikan interface tersebut. Dengan begitu, program menjadi lebih

fleksibel karena satu tipe referensi dapat digunakan untuk berbagai bentuk objek.

5. Percobaan 2 – Virtual method invocation

5. 1. Langkah Percobaan

1. Pada percobaan ini masih akan digunakan class-class dan interface yang digunakan pada percobaan sebelumnya.

2. Buat class baru dengan nama Tester2.

```
package Percobaan2;

import P1.Employee;
import P1.PermanentEmployee;

public class Tester2 {

    public static void main(String[] args) {
        PermanentEmployee pEmp = new PermanentEmployee("Dedik", 500);

        Employee e = pEmp;

        System.out.println(e.getEmployeeInfo());
        System.out.println("--------------------");
        System.out.println(pEmp.getEmployeeInfo());
    }
}
```

3. Jalankan class Tester2, dan akan didapatkan hasil sebagai berikut:

```
Name = Dedik
Registered as permanent employee with salary 500

--------------------
Name = Dedik
Registered as permanent employee with salary 500
```

5.2. Pertanyaan

1. Perhatikan class Tester2 di atas, mengapa pemanggilan e.getEmployeeInfo() pada baris 8 dan pEmp.getEmployeeInfo() pada baris 10 menghasilkan hasil sama?

-Method getEmployeeInfo() pada PermanentEmployee merupakan overriding dari method milik Employee.

Jadi, keduanya menjalankan method yang sama dari PermanentEmployee.

2. Mengapa pemanggilan method e.getEmployeeInfo() disebut sebagai pemanggilan method virtual (virtual method invication), sedangkan pEmp.getEmployeeInfo() tidak?

Karena variabel e bertipe Employee, tetapi objek sebenarnya adalah PermanentEmployee.

Saat program dijalankan, JVM menentukan method berdasarkan jenis objek sebenarnya. Sedangkan pEmp memang sudah bertipe PermanentEmployee, sehingga method yang dipanggil sesuai dengan tipe referensinya.

3. Jadi apakah yang dimaksud dari virtual method invocation? Mengapa disebut virtual?

Virtual method invocation adalah proses pemanggilan overriding method melalui referensi superclass, tetapi method yang dijalankan ditentukan berdasarkan objek sebenarnya saat runtime.Disebut virtual karena method yang dikenali saat compile time bisa berbeda dengan method yang dijalankan saat runtime.

6. Percobaan 3 – Heterogenous Collection

6. 1. Langkah Percobaan

1. Pada percobaan ke-3 ini, masih akan digunakan class-class dan interface pada percobaan sebelumnya.

2. Buat class baru Tester3.

```
cobaan3 > Tester3.java > Tester3 > main(String[])
1   package Percobaan3;
2
3   import P1.ElectricityBill;
4   import P1.Employee;
5   import P1.InternshipEmployee;
6   import P1.Payable;
7   import P1.PermanentEmployee;
8
9   public class Tester3 {
        Run | Debug
0       public static void main(String[] args) {
1           PermanentEmployee pEmp = new PermanentEmployee(name: "Dedik", salary: 500);
2
3           InternshipEmployee iEmp = new InternshipEmployee(name: "Sunarto", length: 5);
4
5           ElectricityBill eBill = new ElectricityBill(kwh: 5, category: "R-1");
6
7           Employee[] e = { pEmp, iEmp };
8           Payable[] p = { pEmp, eBill };
9           Employee[] e2 = { pEmp, iEmp, eBill };
0
```

6.2. Pertanyaan

1. Perhatikan array e pada baris ke-8, mengapa ia bisa diisi dengan objek objek dengan tipe yang berbeda, yaitu objek pEmp (objek dari PermanentEmployee) dan objek iEmp (objek dari InternshipEmployee) ?

- Karena array e bertipe Employee, sehingga bisa menyimpan objek dari subclass Employee, yaitu PermanentEmployee dan InternshipEmployee

2. Perhatikan juga baris ke-9, mengapa array p juga biisi dengan objek-objek dengan tipe yang berbeda, yaitu objek pEmp (objek dari PermanentEmployee) dan objek eBill (objek dari ElectricityBilling) ?

- Karena array p bertipe Payable. Kedua objek tersebut sama-sama mengimplementasikan interface Payable.

3. Perhatikan baris ke-10, mengapa terjadi error?

- Karena eBill merupakan objek dari ElectricityBill, yang tidak memiliki hubungan inheritance dengan Employee.Dengan demikian, objek eBill tidak dapat dimasukkan ke dalam array bertipe Employee.

7. Percobaan 4 – Argumen polimorfisme, instanceof dan casting objek

7.1. Langkah Percobaan

1. Percobaan 4 ini juga masih menggunakan class-class dan interface yang digunakan pada percobaan sebelumnya. 4. Buat class baru dengan nama Owner. Owner bisa melakukan pembayaran baik kepada pegawai permanen maupun rekening listrik melalui method pay(). Selain itu juga bisa menampilkan info pegawai permanen maupun pegawai magang melalui method showMyEmployee().

```java
package Percobaan4;

import P1.ElectricityBill;
import P1.Employee;
import P1.Payable;
import P1.PermanentEmployee;

public class Owner {

    public void pay(Payable p) {
        System.out.println(
                "Total payment = " + p.getPaymentAmount());

        if (p instanceof ElectricityBill) {
            ElectricityBill eb = (ElectricityBill) p;
            System.out.println("" + eb.getBillInfo());
        } else if (p instanceof PermanentEmployee) {
            PermanentEmployee pe = (PermanentEmployee) p;
            pe.getEmployeeInfo();
            System.out.println(pe.getEmployeeInfo());
        }
    }

    public void showMyEmployee(Employee e) {
        System.out.println("" + e.getEmployeeInfo());
        if (e instanceof PermanentEmployee) {
            System.out.println(
                    "You have to pay her/him monthly!");
        } else {
            System.out.println("No need to pay her/him!");
    }
}
```

2. Buat class baru Tester4.

```java
package Percobaan4;

import P1.ElectricityBill;
import P1.InternshipEmployee;
import P1.PermanentEmployee;

public class Tester4 {
    Run | Debug
    public static void main(String[] args) {
        Owner ow = new Owner();

        ElectricityBill eBill = new ElectricityBill(kwh: 5, category: "R-1");

        ow.pay(eBill);
        System.out.println("--------------------");

        PermanentEmployee pEmp = new PermanentEmployee(name: "Dedik", salary: 500);

        ow.pay(pEmp);
        System.out.println("--------------------");

        InternshipEmployee iEmp = new InternshipEmployee(name: "Sunarto", length: 5);

        ow.showMyEmployee(pEmp);
        System.out.println("--------------------");

        ow.showMyEmployee(iEmp);

        // ow.pay(iEmp); // Jika diaktifkan, terjadi error.
    }
}
```

3. Jalankan class Tester4, dan akan didaptkan hasil sebagai berikut:

```
\jdt_ws\JB_12_4af91e47\bin  Percobaan4.Tester4
Total payment = 500
KWH = 5
Category = R-1(100 per KWH)

--------------------
Total payment = 525
Name = Dedik
Registered as permanent employee with salary 500

--------------------
Name = Dedik
Registered as permanent employee with salary 500

You have to pay her/him monthly!
--------------------
Name = Sunarto
Registered as internship employee for 5 month/s
```

1. Perhatikan class Tester4 baris ke-7 dan baris ke-11, mengapa pemanggilan ow.pay(eBill) dan ow.pay(pEmp) bisa dilakukan, padahal jika diperhatikan method pay() yang ada di dalam class Owner memiliki argument/parameter

bertipe Payable? Jika diperhatikan lebih detil eBill merupakan objek dari ElectricityBill dan pEmp merupakan objek dari PermanentEmployee?

Karena ElectricityBill dan PermanentEmployee sama-sama mengimplementasikan interface Payable.Oleh karena itu, kedua objek tersebut dapat diterima sebagai argument method pay().

2. Jadi apakah tujuan membuat argument bertipe Payable pada method pay() yang ada di dalam class Owner?

- Tujuannya agar method pay() bisa menerima berbagai jenis objek yang mengimplementasikan Payable.Jadi, method tersebut tidak perlu dibuat terpisah untuk pembayaran pegawai permanen dan rekening listrik.

3. Coba pada baris terakhir method main() yang ada di dalam class Tester4 ditambahkan perintah ow.pay(iEmp);

```
package Percobaan4;

import P1.ElectricityBill;
import P1.InternshipEmployee;
import P1.PermanentEmployee;

public class Tester4 {
    Run | Debug
    public static void main(String[] args) {
        Owner ow = new Owner();

        ElectricityBill eBill = new ElectricityBill(kwh: 5, category: "R-1");

        ow.pay(eBill);
        System.out.println("--------------------");

        PermanentEmployee pEmp = new PermanentEmployee(name: "Dedik", salary: 500);

        ow.pay(pEmp);
        System.out.println("--------------------");

        InternshipEmployee iEmp = new InternshipEmployee(name: "Sunarto", length: 5);

        ow.showMyEmployee(pEmp);
        System.out.println("--------------------");

        ow.showMyEmployee(iEmp);

        ow.pay(iEmp); // Jika diaktifkan, terjadi error.
    }
}
```

Karena InternshipEmployee tidak mengimplementasikan interface Payable.

4. Perhatikan class Owner, diperlukan untuk apakah sintaks p instanceof ElectricityBill pada baris ke-6 ?

- Digunakan untuk memeriksa apakah objek yang diterima oleh parameter p merupakan objek dari class ElectricityBill.Hasil pemeriksaan berupa boolean, yaitu true atau false.

5. Perhatikan kembali class Owner baris ke-7, untuk apakah casting objek disana (ElectricityBill eb = (ElectricityBill) p) diperlukan ? Mengapa objek p yang bertipe Payable harus di-casting ke dalam objek eb yang bertipe ElectricityBill ?

- Karena variabel p bertipe Payable, sedangkan kita ingin mengakses method khusus dari ElectricityBill, yaitu getBillInfo().Maka dilakukan downcasting agar objek tersebut dapat diperlakukan sebagai ElectricityBill.

8. Tugas

Dalam suatu permainan, Zombie dan Barrier bisa dihancurkan oleh Plant dan bisa menyembuhkan diri. Terdapat dua jenis Zombie, yaitu Walking Zombie dan Jumping Zombie. Kedua Zombie tersebut memiliki cara penyembuhan yang berbeda, demikian juga cara penghancurannya, yaitu ditentukan oleh aturan berikut ini:

• Pada WalkingZombie o Penyembuhan : Penyembuhan ditentukan berdasar level zombie yang bersangkutan §§ Jika zombie level 1, maka setiap kali penyembuhan, health akan bertambah 20% §§ Jika zombie level 2, maka setiap kali penyembuhan, health akan bertambah 30% §§ Jika zombie level 3, maka setiap kali penyembuhan, health akan bertambah 40% o Penghancuran : setiap kali penghancuran, health akan berkurang 2% • Pada Jumping Zombie o Penyembuhan : Penyembuhan ditentukan berdasar level zombie yang bersangkutan §§ Jika zombie level 1, maka setiap kali penyembuhan, health akan bertambah 30% §§ Jika zombie level 2, maka setiap kali penyembuhan, health akan bertambah 40% §§ Jika zombie level 3, maka setiap kali penyembuhan, health akan bertambah 50% o Penghancuran : setiap kali penghancuran, health akan berkurang 1% Buat program dari class diagram di bawah ini!

```java
public class Barrier implements Destroyable {
    private int strength;

    public Barrier(int strength) {
        this.strength = strength;
    }

    public void setStrength(int strength) {
        this.strength = strength;
    }

    public int getStrength() {
        return strength;
    }

    public void destroyed() {
        strength -= 1;

        if (strength < 0) {
            strength = 0;
        }
    }

    public String getBarrierInfo() {
        return "Barrier-Strength:" + strength;
    }
}
```

```java
public interface Destroyable {
    public abstract void destroyed();
}
```

```java
public class JumpingZombie extends Zombie{
    int health = 100;
    public JumpingZombie(int health , int level){
        super(health, level);
    }

    public void heal(){
        if (level==1){
            health = health + (health * 30/100);
        } else if (level==2){
            health = health + (health * 40/100);
        }else if (level==3){
            health = health + (health * 50/100);
        }

        }

    @Override
    public void destroyed(){
         health = health - (health * 1/100);
        }

     public String getZombieInfo(){
        return  "Jumping Zombie Data" +
        "\nHealth = " + health + "\nLevel = " + level;
    }
    }
```

```java
public class Plant {
    public void doDestroy(Destroyable d) {
        d.destroyed();
    }
}
```

```java
public class Tester {
    Run | Debug
    public static void main(String[] args) {
        WalkingZombie wz = new WalkingZombie(health: 100, level: 1);
        JumpingZombie jz = new JumpingZombie(health: 100,level: 2);
        Barrier b = new Barrier(strength: 100);
        Plant p = new Plant();
        System.out.println("" +wz.getZombieInfo());
        System.out.println("" +jz.getZombieInfo());
        System.out.println("" +b.getBarrierInfo());
        System.out.println("--------------------");
        for(int i=0; i<4; i++){
            p.doDestroy(wz);
            p.doDestroy(jz);
            p.doDestroy(b);
        }
        System.out.println("" +wz.getZombieInfo());
        System.out.println("" +jz.getZombieInfo());
        System.out.println("" +b.getBarrierInfo());

    }
}
```

```java
public class WalkingZombie extends Zombie {

    int health = 100;

    public WalkingZombie(int health, int level) {
        super(health, level);
    }

    public void heal() {
        if (level == 1) {
            health = health + (health * 10 / 100);
        } else if (level == 2) {
            health = health + (health * 30 / 100);
        } else if (level == 3) {
            health = health + (health * 40 / 100);
        }
    }

    @Override
    public void destroyed() {
        health = health - (health * 2 / 100);
    }

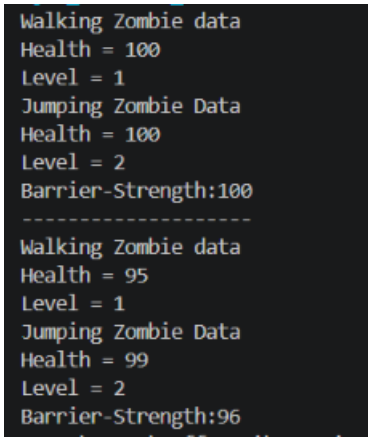
     public String getZombieInfo(){
        return  "Walking Zombie data" +
        "\nHealth = " + health + "\nLevel = " + level;
    }
}
```

```java
public abstract class Zombie implements Destroyable{

    protected int health;
    protected int level;

    public Zombie (int health, int level){
        this.health = health;
        this.level = level;
    }

    public abstract void heal();
    public abstract void destroyed();
    public String getZombieInfo(){
        return  "zombie data" +
        "\nHealth = " + health + "\nLevel = " + level;
    }
}
```


Walking Zombie data
Health = 100
Level = 1
Jumping Zombie Data
Health = 100
Level = 2
Barrier-Strength:100

Walking Zombie data
Health = 95
Level = 1
Jumping Zombie Data
Health = 99
Level = 2
Barrier-Strength:96